

# Tarif Rates

## Problem

- Find the cheapest electricity production plan which satisfies the demand, with the given electricity generators.

## Sets

- Plant Types:  $p$
- Hour Periods:  $h$

## Parameters

- Number Generators:  $NG_p$
- Number of hours in period:  $NHP_h$
- Maximal production level per type:  $MAXL_p$
- Minimal production level per type:  $MINL_p$
- Min cost:  $MINC_p$
- Production cost above min:  $PC_p$
- Startup cost:  $STARTC_p$
- Power demand:  $PDEM_h$
- Spare capacity requirement: 15%

## Decision variables

- $x_{p,h}$ : Number of plants  $p$  running in period  $h$ , integer
- $y_{p,h}$ : Power production on plants  $p$  in period  $h$ , positive variable.
- $n_{p,h}$ : Number of plants  $p$  started in period  $h$ , positive variable.

## Model

### Objective:

- Objective is cost to be minimized:

$$\begin{aligned} & \sum_h \sum_p MINC_p \cdot NHP_h \cdot x_{p,h} \\ & + \sum_h \sum_p NHP_h \cdot PC_p \cdot (y_{p,h} - MINL_p \cdot x_{p,h}) \\ & + \sum_h \sum_p STARTC_p \cdot n_{p,h} \end{aligned}$$

### Constraints:

- Satisfy demand:

$$\sum_p y_{p,h} \geq PDEM_h \quad \forall h$$

- Limit production to less than max:

$$y_{p,h} \leq MAXL_p \cdot x_{p,h} \quad \forall p, h$$

- Limit production to more than min:

$$y_{p,h} \geq MINL_p \cdot x_{p,h} \quad \forall p, h$$

- Force startup variable

$$n_{p,h} \geq x_{p,h} - x_{p,h-1} | (h > 1) - x_{p,H} | (h = 1) \quad \forall p, h$$

- Ensure extra 15% extra capacity:

$$\sum_p MAXL_p \cdot x_{p,h} \geq (1 + 0.15) PDEM_h \quad \forall h$$

The full model in Julia/JuMP, available with the name

`TarifRates1.jl`

from the book web-site, is given below:

```

*****
# Tarif Rates Assignment Question 1, "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
# PARAMETERS
Hours = ["0.00-6.00", "6.00-9.00", "9.00-15.00", "15.00-18.00", "18.00-24.00"]
H=length(Hours)
Plants = ["Type1", "Type2", "Type3"]
P=length(Plants)

noplants = [12 10 5]
nohours = [6 3 6 3 6]
demand = [15000 30000 25000 40000 27000]
minlevel = [850 1250 1500]
maxlevel = [2000 1750 4000]
costhouratmin = [1000 2600 3000]
costMWHouraboveMin = [2 1.3 3]
startupcost = [2000 1000 500]
*****

# Model
tarifrates = Model(HiGHS.Optimizer)

@variable(tarifrates, x[p=1:P, h=1:H], Int) # no plants running of each of the three types
for p=1:P for h=1:H set_lower_bound(x[p,h], 0) end end
for p=1:P for h=1:H set_upper_bound(x[p,h], noplants[p]) end end

@variable(tarifrates, y[1:P, 1:H] >= 0) # production of each type running

@variable(tarifrates, s[1:P, 1:H] >= 0) # no started up

# Minimize production cost
@objective(tarifrates, Min,
    sum( nohours[h]*costhouratmin[p]*x[p,h] for p=1:P, h=1:H) + # fixed costs for run
    sum( nohours[h]*costMWHouraboveMin[p]*(y[p,h]-minlevel[p]*x[p,h]) for p=1:P, h=1:H) +
    sum( startupcost[p]*s[p,h] for p=1:P, h=1:H)) # startup costs

# require min level production
@constraint(tarifrates, [p=1:P, h=1:H],
    y[p,h] >= minlevel[p]*x[p,h])

# limit max level production
@constraint(tarifrates, [p=1:P, h=1:H],
    y[p,h] <= maxlevel[p]*x[p,h])

```

```

# demand
@constraint(tarifrates, [h=1:H],
            sum(y[p,h] for p=1:P) >= demand[h])

# force startup costs
@constraint(tarifrates, [p=1:P, h=1:H],
            s[p,h] >= x[p,h] - (h > 1 ? x[p,h-1] : x[p,H]))

# require extra demand robustness
@constraint(tarifrates, [h=1:H],
            sum( maxlevel[p]*x[p,h] for p=1:P) >= 1.15*demand[h])

print(tarifrates)

#####

#####
# solve
optimize!(tarifrates)
println("Termination status: $(termination_status(tarifrates))")
#####

#####
# Report results
println("-----");
if termination_status(tarifrates) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(tarifrates))")
else
    println(" No solution")
end
println("-----");
#####

#####
println("Successful end of $(PROGRAM_FILE)")
#####
println("running: ", value.(x))
println("fixed-cost: ", sum( nohours[h]*costhouratmin[p]*value(x[p,h]) for p=1:P, h=1:H) )
println("var-cost: ", sum( nohours[h]*costMWHouraboveMin[p]*(value(y[p,h])-minlevel[p]*value(s[p,h])) for p=1:P, h=1:H) )
println("start-up cost: ", sum( startupcost[p]*value(s[p,h]) for p=1:P, h=1:H))

```

Comments to the Julia/JuMP program:

- Notice: we do NOT need to model each powerplant with a binary variable,

but we only need to model the number of plants running of a certain type!

- Notice: we use circular "storage", such that the number of generators running in the first period comes from the number running in the last period. This is achieved by the in-line if statement:

$$(h > 1 \text{ ? } x[p, h-1] : x[p, H])$$

- Notice: it is simple to find the cost of the 15 % extra power production buffer: Simply remove, or comment out, the demand buffer constraint

The full model for question 2 in Julia/JuMP, is available with the name

TarifRates2.jl

from the book web-site, is given below:

```
#####
# Tarif Rates Assignment Question 2, "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
# PARAMETERS
Hours = ["0.00-6.00", "6.00-9.00", "9.00-15.00", "15.00-18.00", "18.00-24.00"]
H=length(Hours)
Plants = ["Type1", "Type2", "Type3"]
P=length(Plants)

noplants = [12 10 5]
nohours = [6 3 6 3 6]
demand = [15000 30000 25000 40000 27000]
minlevel = [850 1250 1500]
maxlevel = [2000 1750 4000]
costhouratmin = [1000 2600 3000]
costMWHouraboveMin = [2 1.3 3]
startupcost = [2000 1000 500]
#####

#####
# Model
tarifrates = Model(HiGHS.Optimizer)

@variable(tarifrates, x[p=1:P, h=1:H], Int) # no plants running of each of the three types
for p=1:P for h=1:H set_lower_bound(x[p,h], 0) end end
for p=1:P for h=1:H set_upper_bound(x[p,h], noplants[p]) end end

@variable(tarifrates, y[1:P, 1:H] >= 0) # production of each type running
```

```

@variable(tarifrates, s[1:P, 1:H] >= 0) # no started up

# Minimize production cost
@objective(tarifrates, Min,
    sum( nohours[h]*costhouratmin[p]*x[p,h] for p=1:P, h=1:H) + # fixed costs for run
    sum( nohours[h]*costMWHouraboveMin[p]*(y[p,h]-minlevel[p]*x[p,h]) for p=1:P, h=1:H) +
    sum( startupcost[p]*s[p,h] for p=1:P, h=1:H)) # startup costs

# require min level production
@constraint(tarifrates, [p=1:P, h=1:H],
    y[p,h] >= minlevel[p]*x[p,h])

# limit max level production
@constraint(tarifrates, [p=1:P, h=1:H],
    y[p,h] <= maxlevel[p]*x[p,h])

# demand
@constraint(tarifrates, [h=1:H],
    sum(y[p,h] for p=1:P) >= demand[h])

# force startup costs
@constraint(tarifrates, [p=1:P, h=1:H],
    s[p,h] >= x[p,h] - (h > 1 ? x[p,h-1] : x[p,H]))

print(tarifrates)
#####

#####
# solve
optimize!(tarifrates)
println("Termination status: $(termination_status(tarifrates))")
#####

#####
# Report results
println("-----");
if termination_status(tarifrates) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(tarifrates))")
else
    println(" No solution")
end
println("-----");
#####

```

```
#####  
println("Successfull end of $(PROGRAM_FILE)")  
#####
```